

Assignment 2 – Student Performance Analysis System

Total Marks: 20

Objective

To design and implement a Python program that reads and analyses student performance data across multiple campuses, identifies low-performing students, generates summary reports, and allows keyword-based searching in an academic report file.

Case Study Description

A university manages student performance data separately for each campus. Each campus maintains a text file named in the following format:

CAMPUS_CODE.txt

Examples:

CMP001.txt, CMP002.txt

Each line in the file represents one student's marks in four key subjects and their unique student ID, structured as follows:

Programming, Database, Systems-Analysis, Networking, StudentID

Example line:

78.0 84.0 79.5 85.0 s98765

Integrated Program Requirements

Write a single Python program that performs the following tasks together:

1. Load and Process Campus Files

- Read all campus text files in the current working directory.
- For each campus, calculate the average marks for:
 - Programming
 - Database Systems
 - Systems Analysis
 - Networking

2. Identify Low-Performing Students

- Compute the overall average for each student using equal weight for all subjects.
- If the total average is below 50, mark the student as "At Risk."

3. Generate Campus Summary File

- For each campus, create one summary file named: CAMPUS_CODE_summary.txt

Example content:

Campus Code: CMP001

Average Programming Mark: 80.5

Average Database Mark: 85.2

Average Systems Analysis Mark: 78.6

Average Networking Mark: 83.4

Students At Risk (Average < 50):

s98234

s97321

4. Keyword Search Feature

Create a file named: academic_performance_report.txt

This report must include:

- Purpose of the analysis
- Total number of campuses processed
- Total students processed
- Overall subject trends
- Institute-wide high-performer percentage
- Institute-wide at-risk percentage
- Recommendations

- After processing, open the text file academic_performance_report.txt.

- Continuously prompt the user to enter a keyword.

- Count and display how many times the keyword appears (case-insensitive).

- Stop when the user types "exit."

Example:

Enter keyword to search: average

Keyword 'average' found 4 times.

Enter keyword to search: exit

Exiting search...

5. Submission Requirements:

- Submit the **Python code** along with a **report** explaining the implementation and expected output.
(Zip the code and report together)
- The **report** should include an explanation of the logic, how different test cases were handled, and **sample output files**.
- The **cover page of the report is attached**.